

METAPUSHER: Meta Learning and Planning for Nonprehensile Manipulation of Unseen Objects with Rapid Online Adaption

Donghyung Lee*, Seyedali Golestaneh*, Jaskrit Singh, Zhuoyun Zhong,
Athanasios Kapoutsis, and Constantinos Chamzas

Abstract—Manipulating previously unseen objects remains challenging, as their dynamics depend on latent physical properties, such as friction and mass distribution, that cannot be inferred from perception alone. Prior experience across objects can provide an initial estimate of unseen object dynamics, but this estimate remains uncertain and can degrade further during sim-to-real transfer. Adapting the dynamics through interaction can progressively refine the estimation, however, updating the model may invalidate the planned trajectory. Successful and efficient manipulation therefore requires both rapid dynamics adaptation and a planning strategy that can incorporate this evolution. In this work, we introduce METAPUSHER, a meta-learning and adaptive planning framework for nonprehensile manipulation of unseen objects without prior object-specific interactions. A meta-learned dynamics model rapidly adapts from interactions during task execution, while an adaptive kinodynamic planner updates long-horizon plans by reusing and refining its existing search tree. This coupling enables manipulation and adaptation without a separate data collection phase. We evaluate METAPUSHER on unseen objects in simulation and in sim-to-real scenarios, comparing against fine-tuning and active learning methods, MPPI-based control, and a reinforcement learning policy. It achieves lower prediction error and improves task success rate by up to 20%. The source code is available at <https://anonymous.4open.science/r/metapusher-B2BB/README.md>

I. INTRODUCTION

Nonprehensile manipulation of previously unseen objects is challenging because pushing outcomes depend not only on the object’s geometry but also on its latent physical properties, such as friction, mass distribution, and inertia. Analytical models provide useful structure for planar pushing [1], [2], while data-driven methods learn the dynamics from collected interaction data [3]–[5]. However, in a more general setting, when a robot encounters a new object, it should be able to begin manipulation without prior object-specific interactions.

Existing methods address parts of this problem separately. Generalizable manipulation methods [6], [7] transfer experience across objects from geometric features, but this information alone cannot capture the full object behavior. In contrast, adaptive methods update dynamics from interactions [8]–[11] but typically rely on dedicated data collection and short-horizon control. Kinodynamic planning methods [12]–[14] provide long-horizon reasoning, but their trajectories are generated under a fixed forward model and may no longer be dynamically feasible when that model evolves. Therefore, efficiently manipulating unseen objects requires adapting both the dynamics model and the plan during task execution.

In this paper, we introduce METAPUSHER, a meta-learning and adaptive planning framework for manipulation of unseen objects with no prior interactions. METAPUSHER uses MAML

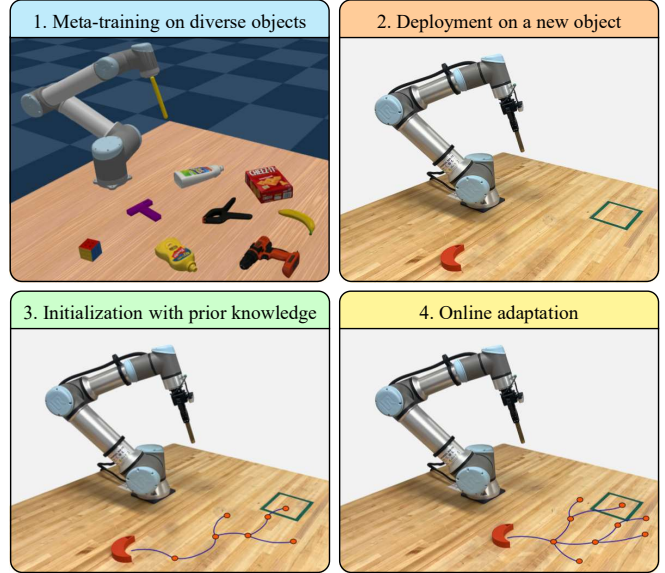


Fig. 1: **METAPUSHER Overview:** (1) An initial forward dynamics model is meta-trained across diverse objects in simulation; (2) At deployment, a previously unseen object is encountered for the first time, with zero prior interactions; (3) The manipulation task begins directly using the initial dynamics model; (4) Executed pushes adapt both the dynamics model and the existing planning tree under the evolving dynamics.

[15] to learn an adaptive dynamics model across diverse objects. At deployment, an adaptive kinodynamic planner uses the dynamics estimate to generate a long-horizon trajectory. After each executed push, the dynamics model adapts based on the recently observed transitions, and the adaptive kinodynamic planner prunes and refines the existing search tree under the updated model rather than replanning from scratch. This couples online dynamics adaptation with long-horizon adaptive planning and requires no separate object-specific data collection phase.

Our main contributions are:

- A geometry-conditioned meta-learning dynamics model that enables manipulation of unseen objects without prior object-specific interactions and rapidly adapts from recent transitions during task execution.
- An adaptive kinodynamic planning strategy that retains and updates the existing search tree as the dynamics model updates.
- Extensive simulation and real-world evaluation across a total of 14 unseen objects, demonstrating effective generalization and sim-to-real transfer, including a **100%** task success rate in the real world.

II. RELATED WORK

This section reviews the prior work most relevant to our method from three perspectives: (i) generalizable nonprehensile manipulation, (ii) online dynamics adaptation, (iii) long-horizon kinodynamic planning.

A. Generalizable Nonprehensile Manipulation

Analytical models of nonprehensile pushing have been developed to enable planar manipulation planning [1], [2]. However, in practice, accurate prediction remains difficult since pushing outcomes depend on geometry, contact, friction, and mass distribution [4]. Data-driven methods therefore learn planar pushing dynamics directly from interaction data [5], [16], while methods like ActivePusher [3] improve data efficiency through uncertainty-guided data acquisition. Such approaches can provide accurate models, but require dedicated object-specific interaction data to characterize the dynamics of an unseen object.

A complementary line of work learns across diverse objects using simulation interactions and transfers prior experience to previously unseen objects using visual or geometric representations. HACMan [6], [17], CORN [7] learn point-cloud-based policies for zero-shot sim-to-real transfer and unseen object generalizability, while Push Anything [18] uses contact implicit MPC using geometry reconstruction. Although these methods generalize well to unseen objects, observable geometry alone cannot capture latent physical properties that govern object motion. In contrast, METAPUSHER begins the manipulation directly from geometric observations and progressively adapts the dynamics online using interactions generated during task execution, without requiring a separate object-specific data collection phase.

B. Online Dynamics Adaptation

Online adaptation estimates dynamics model information during interaction. Meta-learning enables rapid specialization from limited experience, with MAML [15] learning an initialization that can be adapted with only a few gradient updates. GrBAL [8] exploits this principle and learns a meta-learned dynamics that updates from recent transitions, while [19] meta-learns parameters for real-time online dynamics adaptation in off-road autonomous driving. HyperDynamics [9] specializes object dynamics after acquiring a small number of interactions with a new object before starting the manipulation. Push-Net [10] recurrently uses pushing history to reason about unknown physical properties, RMPPI [20] uses recurrent forward dynamics to adapt simulation-trained models to real object behavior, and DyWA [11] combines manipulation history with point cloud geometry for sim-to-real adaptation. Unlike prior methods that primarily require separate interactions, METAPUSHER learns an adaptive forward dynamics model based on geometry, begins goal directed manipulation with zero object specific data, and specializes the model through task interactions. Moreover, instead of short-horizon control or repeated MPC [21], METAPUSHER incorporates the evolving dynamics into long-horizon kinodynamic planning.

This enables instant manipulation of unseen objects while adapting both the dynamics model and the plan.

C. Long-Horizon Kinodynamic Planning

Most adaptive dynamics approaches above employ receding-horizon optimization, including MPC and MPPI variants [22]. Although these controllers can incorporate updated dynamics at each cycle, their finite-horizon optimization can lead to myopic behavior. Sampling-based kinodynamic planners [12] instead search directly over dynamically feasible trajectories and have been applied to long-horizon manipulation, as in [3]. KRAFT [13] and AURA [14] reuse prior planning information during execution to improve robustness and replanning efficiency. Meanwhile, recent GPU-parallelized planners, such as Kino-PAX [23] and PAKR [24], reduce planning time substantially through parallel tree expansion. These methods, however, assume a fixed dynamics model during planning and do not address the case where the forward model changes. METAPUSHER instead couples online dynamics adaptation with long-horizon kinodynamic planning by retaining and updating the existing search tree as the learned dynamics evolve, rather than planning from scratch under a fixed model.

III. PROBLEM STATEMENT

We consider planar pushing manipulation of a previously unseen object with unknown dynamics. Let $\mathcal{X}$ and $\mathcal{U}$ denote the object's state space and control space, respectively. The state space $\mathcal{X}$ is partitioned into two disjoint subsets, $\mathcal{X} = \mathcal{X}_{\text{obs}} \cup \mathcal{X}_{\text{free}}$, where $\mathcal{X}_{\text{obs}}$ represents the invalid state region and $\mathcal{X}_{\text{free}}$ is the valid state region.

Let $\mathcal{O}_{\text{train}}$ denote the set of objects available during training and $\mathcal{O}_{\text{test}}$ the set used for evaluation, where $\mathcal{O}_{\text{train}} \cap \mathcal{O}_{\text{test}} = \emptyset$. For each $o \in \mathcal{O}_{\text{train}}$, arbitrary interaction data $\mathcal{D}_o^{\text{train}} = \{(x_j, u_j, x_{j+1})\}_{j=1}^M$ may be collected during training in simulation, whereas each $o \in \mathcal{O}_{\text{test}}$ is encountered for the first time at deployment with zero prior interaction data $\mathcal{D}_o^{\text{test}} = \emptyset$. The result of a push on object o is governed by an unknown transition function:

$$x_{k+1}^{\text{gt}} = \Gamma_o^{\text{gt}}(x_k^{\text{gt}}, u_k), \quad (1)$$

where x_k^{gt} is the state at iteration k , and x_{k+1}^{gt} is the ground-truth state after applying u_k .

Problem Definition: For an object $o \in \mathcal{O}_{\text{test}}$ with an initial state $x_{\text{start}} \in \mathcal{X}_{\text{free}}$, the manipulation objective is to find the shortest feasible trajectory $\pi = (x_0, u_0, \dots, x_{T-1}, u_{T-1}, x_T)$ that drives the object to the goal region $\mathcal{X}_{\text{goal}} \subseteq \mathcal{X}_{\text{free}}$, such that:

$$\begin{aligned} x_{i+1} &= \Gamma_o^{\text{gt}}(x_i, u_i), & x_0 &= x_{\text{start}}, & x_T &\in \mathcal{X}_{\text{goal}}, \\ x_{0:T} &\in \mathcal{X}_{\text{free}}, & u_{0:T-1} &\in \mathcal{U}. \end{aligned} \quad (2)$$

In the next section, we present METAPUSHER, our approach for addressing this problem.

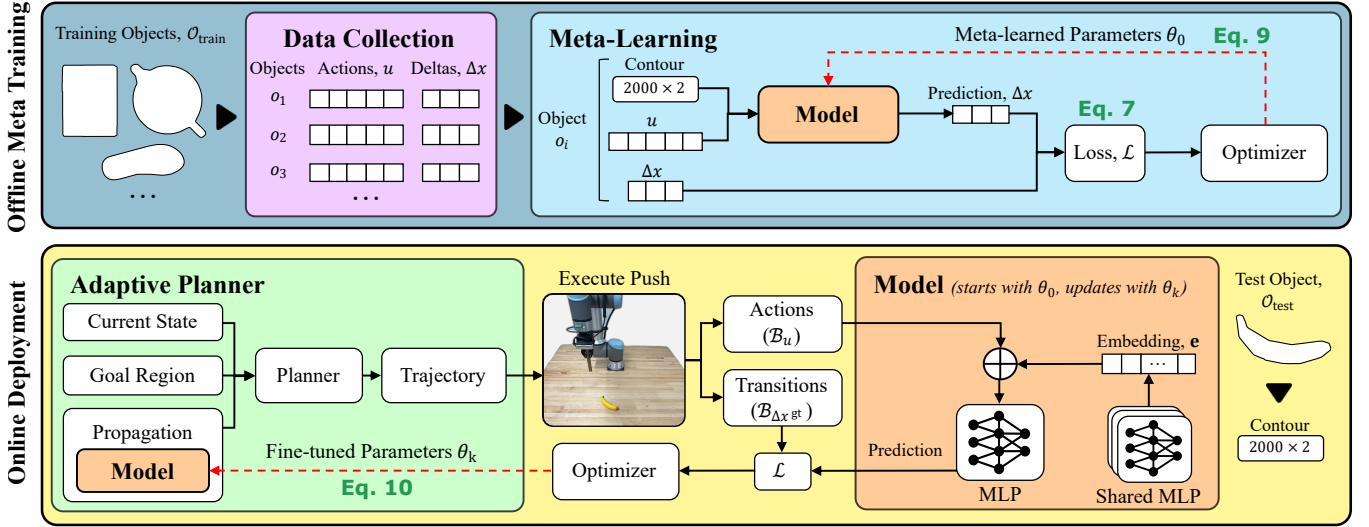


Fig. 2: **METAPUSHER Framework**. During *Offline Meta Training*, interaction data from training objects are used to learn the initialization θ_0 of the geometry-informed forward dynamics model. At *Online Deployment*, the planner uses the current dynamics model to generate a trajectory and execute a push. The resulting transition is then used to update the model parameters to θ_k . The adapted model is then returned to the planner to update the search tree, allowing the dynamics model and planned trajectory to evolve together while manipulating a previously unseen object.

IV. METHODOLOGY

Our method consists of two phases: *Offline Meta Training* and *Online Deployment*. During offline training, interaction data $\mathcal{D}_o^{\text{train}}$ from objects $o \in \mathcal{O}_{\text{train}}$ are used to learn an initial geometry-encoded forward dynamics model (Sec. IV-A). During online deployment, an object $o \in \mathcal{O}_{\text{test}}$ is encountered with no prior interactions, and the initial forward dynamics model is used to start the manipulation. The model parameters are continuously adapted using each observed transition (Sec. IV-B), while an adaptive planning strategy updates the existing planning tree to remain consistent with the evolving dynamics (Sec. IV-C). Fig. 2 summarizes the complete framework.

Push actions are defined from the object’s local 2D contour point cloud, g_o . We select a contact position $[c_x, c_y]$ on the contour and apply the push along the inward surface normal at the contact point, with angle ϕ . To avoid the angular discontinuity at $\pm\pi$, the pushing angle is represented by $[\sin \phi, \cos \phi]$. Although c_x and c_y fully determine ϕ , we found explicitly including it to be beneficial for training. With a push distance $d \in [d_{\min}, d_{\max}]$, each push action is represented as:

$$u = [c_x, c_y, \sin \phi, \cos \phi, d]. \quad (3)$$

To incorporate object geometry into the dynamics model, g_o is encoded using a PointNet-style architecture [25], extracting an 1024-dimensional global feature vector $\mathbf{e} = [e_1, \dots, e_{1024}]$. This embedding provides a global representation of the object’s contour geometry and is concatenated with the push action and provided directly to the forward dynamics model:

$$[c_x, c_y, \sin \phi, \cos \phi, d, e_1, \dots, e_{1024}]^T \in \mathbb{R}^{1029}. \quad (4)$$

Then, the learned forward dynamics model uses this concatenated representation to predict the resulting planar object displacement, represented in $SE(2)$ as:

$$\Delta x = [\Delta p_x, \Delta p_y, \Delta \psi]. \quad (5)$$

A. Offline Meta Training

Each $o \in \mathcal{O}_{\text{train}}$ defines an individual meta-learning objective. Using the corresponding interaction data from $\mathcal{D}_o^{\text{train}}$, we seek a general initialization θ_0 for the forward dynamics model Γ^θ that captures transferable pushing behavior to $\mathcal{O}_{\text{test}}$. Conditioned on the object geometry g_o and push action, Γ^θ predicts the state transition:

$$x_{k+1} = \Gamma_\theta(x_k, u_k; g_o). \quad (6)$$

The learning objective is a weighted mean squared error between the predicted and observed planar displacement:

$$\mathcal{L} = \|\Delta p - \Delta p^{\text{gt}}\|_2^2 + w_\psi \|\Delta \psi - \Delta \psi^{\text{gt}}\|_2^2, \quad (7)$$

where $\Delta p = [\Delta p_x, \Delta p_y]^T$ is the translational displacement, and $w_\psi = 0.2$ is the weight of the rotational displacement error.

We train our model using a meta-learning framework, following Model-Agnostic Meta-Learning (MAML) [15]. Offline Meta Training is performed across $\mathcal{O}_{\text{train}}$ with only a small amount of data per object for adaptation, matching the deployment setting in which only limited interaction data become available for unseen objects.

For each training object o_i , a support set $\mathcal{D}_{o_i}^{\text{support}}$ and a query set $\mathcal{D}_{o_i}^{\text{query}}$ are sampled from the interaction data $\mathcal{D}_{o_i}^{\text{train}}$ and used for per-object adaptation and evaluation of the adapted model within meta-learning, respectively, such that:

$$\begin{aligned} \mathcal{D}_{o_i}^{\text{support}} \cup \mathcal{D}_{o_i}^{\text{query}} &\subseteq \mathcal{D}_{o_i}^{\text{train}}, \\ \mathcal{D}_{o_i}^{\text{support}} \cap \mathcal{D}_{o_i}^{\text{query}} &= \emptyset. \end{aligned} \quad (8)$$

The meta-learned parameters are then updated across the training objects as:

$$\theta \leftarrow \theta - \beta \sum_i \nabla_\theta \mathcal{L}(\theta - \underbrace{\alpha \nabla_\theta \mathcal{L}(\theta, \mathcal{D}_{o_i}^{\text{support}})}_{\theta'_{o_i}, \text{ Inner Gradient Step}}, \mathcal{D}_{o_i}^{\text{query}}), \quad (9)$$

where α is the inner-loop adaptation learning rate, β is the outer-loop meta adaptation learning rate, and $\mathcal{L}(\theta, \mathcal{D})$ denotes the loss defined in Eq. 7. The inner gradient step adapts the model parameters to each object using $\mathcal{D}_{o_i}^{\text{support}}$, while the outer loop evaluates the object-specific parameters on $\mathcal{D}_{o_i}^{\text{query}}$ and backpropagates the resulting loss to update the shared parameters θ accordingly. This explicitly encourages θ to serve as an initialization from which the model can rapidly adapt to the pushing dynamics of a new object.

B. Online Adaptation

For a previously unseen object $o \in \mathcal{O}_{\text{test}}$, deployment begins using $\Gamma_{\theta_0}(\cdot, \cdot)$ with no prior interactions. The initial dynamics model therefore relies on the parameters transferred from $\mathcal{O}_{\text{train}}$ together with the observed object geometry g_o . After executing each push u_k , the action and the observed transition Δx^{gt} from x_k^{gt} to x_{k+1}^{gt} are stored on $\mathcal{B}_u$ and $\mathcal{B}_{\Delta x^{\text{gt}}}$, respectively, and used to update the model parameters from θ_k to θ_{k+1} .

Let $\mathcal{D}_{o,k}^{\text{test}}$ denote the interaction data available for object o after k iteration, assuming $\mathcal{D}_{o,0}^{\text{test}} = \emptyset$. From the current parameters θ_k , online adaptation follows the same gradient update used in meta-learning of Eq. 9, but is now performed using transitions generated during the ongoing manipulation task:

$$\theta_{k+1} = \theta_k - \underbrace{\alpha \nabla_{\theta_k} \mathcal{L}(\theta_k, \mathcal{D}_{o,k}^{\text{test}})}_{\text{Online Gradient Update}}. \quad (10)$$

The updated model $\Gamma_{\theta_{k+1}}$ is then used by the adaptive planner to update the current trajectory, allowing the dynamics model and long-horizon trajectory to evolve together as additional interactions become available.

C. Adaptive Planning

METAPUSHER uses a sampling-based kinodynamic planner to generate long-horizon dynamically feasible pushing trajectories under the adaptive dynamics model. At execution iteration k , given the current state x_k^{gt} and current dynamics $f_o^{\theta_k}$, the planner computes a trajectory $\pi_k = (x_0, u_0, \dots, x_{T-1}, u_{T-1}, x_T)$, such as:

$$\begin{aligned} x_0 &= x_k^{\text{gt}}, & x_T &\in \mathcal{X}_{\text{goal}}, & x_{i+1} &= x_i \oplus f_o^{\theta_k}(x_i, u_i), \\ x_{0:T} &\in \mathcal{X}_{\text{free}}, & u_{0:T-1} &\in \mathcal{U}, \end{aligned} \quad (11)$$

where $\oplus$ denotes the pose composition operator in $SE(2)$.

After executing each control and observing the resulting state x_{k+1}^{gt} , the online adaptation process, described in Sec. IV-A, changes the dynamics model from $f_o^{\theta_k}$ to $f_o^{\theta_{k+1}}$. Consequently, the search tree propagated under the previous model may become inconsistent with the updated dynamics. Moreover, prediction discrepancies can accumulate through its descendants. To address this, the proposed adaptive planner retains and updates the existing search tree after each model update rather than replanning from scratch.

Fig. 3 shows the search tree available at execution iteration k , $\mathcal{T}_k$, where vertices denote predicted states and edges represent the push actions. After selecting and executing the

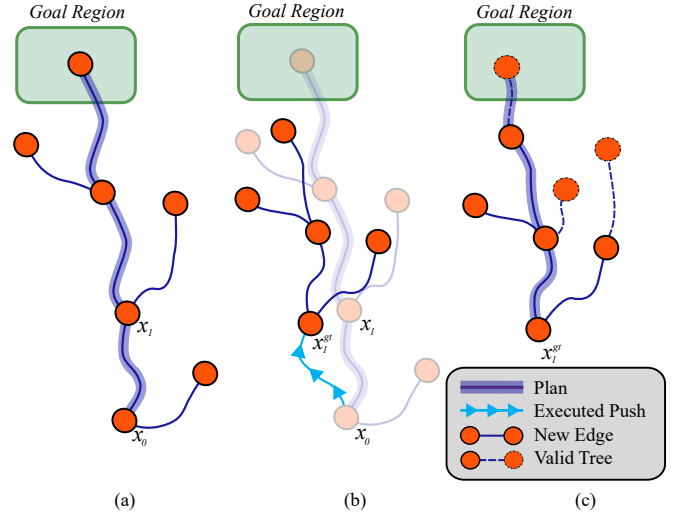


Fig. 3: **Adaptive Planning.** (a) Initial planning tree searching for a path to the goal region. (b) After the first push, we arrive at x_1^{gt} instead of x_1 , so we update our pushing model accordingly. We use the new pushing model to reconstruct the old tree. (c) If the reconstructed tree no longer reaches the goal region, we expand the tree until a new path to the goal is found.

next action u_0 from the current solution, the resulting state x_{k+1}^{gt} is observed and used to adapt the model to $f_o^{\theta_{k+1}}$. The planning tree is then adapted through three operations: *Prune*, *Refinement*, and *Expansion*.

Prune: After executing u_0 , the branches that are no longer descendants of x_1 are not dynamically reachable and are removed. This strategy retains a subtree that preserves search exploration that remains valid from the new state.

Refinement: The planner then sets the new root at the observed state x_1^{gt} , and reconstructs the remaining subtree under $f_o^{\theta_{k+1}}$, proceeding from the root toward the leaves so that the updated plan is consistent through all the states. The planner checks the validity of vertices of the new tree $\mathcal{T}_{k+1}$ and removes any vertex propagated into $\mathcal{X}_{\text{obs}}$.

Expansion: If the refinement invalidates the previous goal-reaching trajectory, the planner resumes the search from $\mathcal{T}_{k+1}$ and adds new nodes propagated using $f_o^{\theta_{k+1}}$. This allows METAPUSHER to leverage previous exploration while improving solution accuracy under the updated dynamics estimate.

This process is repeated after each object interaction. Therefore, METAPUSHER couples two forms of adaptation by updating an unknown object's dynamics while refining the long-horizon plan.

Overall, METAPUSHER integrates meta-learning and online dynamics adaptation with adaptive kinodynamic planning within a unified manipulation framework. The meta-learned model provides transferable estimates for previously unseen objects. Meanwhile, the planning tree adapts as the dynamics evolve, retaining previous planning information while staying consistent with the updated model. In the next section, we evaluate the performance of different aspects of the proposed method.

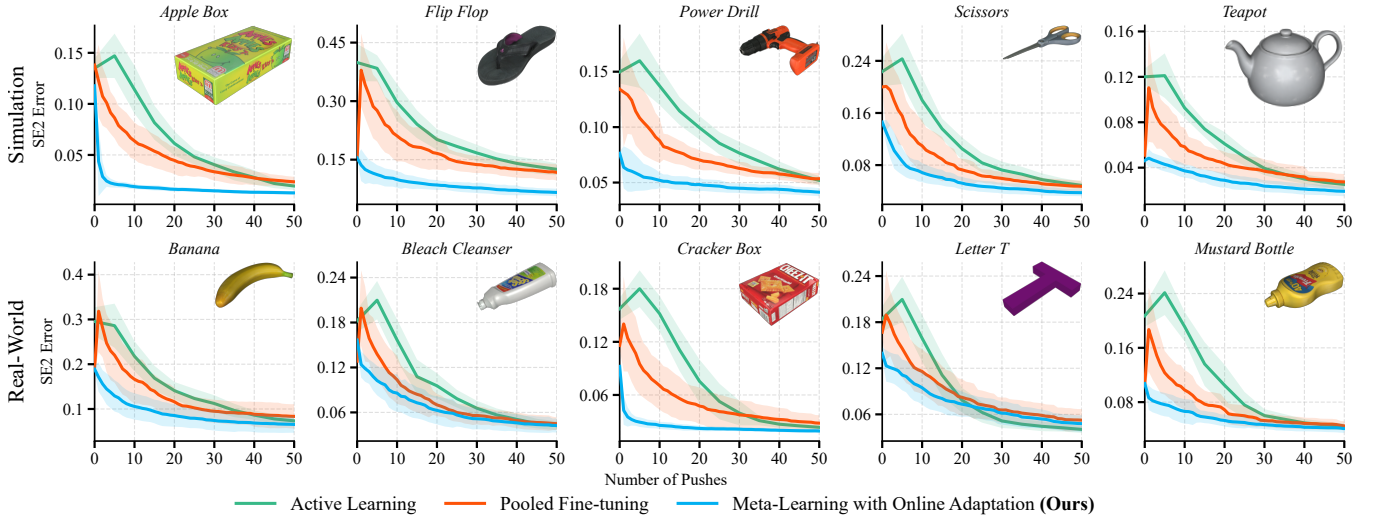


Fig. 4: **Adaptation Performance on Five Simulated and Five Physical Objects.** Prediction error in $SE(2)$ is evaluated on a held-out set, as additional object interactions are used for adaptation (in batches of 10 for ACTIVE LEARNING). For each method, curves are averaged across 30 independent trials, with shaded regions representing one standard deviation.

V. EXPERIMENTS

We evaluate METAPUSHER through both dynamics adaptation and manipulation tasks in both simulation and real-world settings. Sec. V-A investigates how efficiently the meta-learned dynamics model adapts to previously unseen objects using a few interactions, and Sec. V-B evaluates how the proposed framework performs in a manipulation task. Moreover, Sec. V-C investigates the effect of changing the training set $\mathcal{O}_{\text{train}}$.

We train the dynamics models in simulation using 238 objects drawn from the YCB Object dataset [26] and Google Scanned Objects [27], collecting 2000 simulated pushing transitions for each. For planning, we use AORRT [28]¹ implemented in OMPL [29]. Our setup includes a 6-Dof UR16e robot equipped with a rigid cylindrical pushing end effector. For the physical setup, we use the Foundation-Pose model [30] to estimate object pose with an INTEL REALSENSE D435 depth camera mounted on the end effector.

Planning is performed in $\mathcal{X}_{\text{free}} \subseteq SE(2)$, illustrated in Fig. 5 together with the start and goal regions. The object starts at a randomly sampled pose within the start region, and a successful trial is when the object’s Oriented Bounding Box (OBB) lies within the goal region. METAPUSHER is trained according to Sec. IV-A across the $\mathcal{O}_{\text{train}}$ and is compared against the following baselines:

• Learning Baselines

- **POOLED FINE-TUNING:** Uses the same network architecture, but is trained by pooling all the data from $\mathcal{O}_{\text{train}}$ under a conventional supervised learning objective. This method excludes the meta-learning upside.
- **ACTIVE LEARNING:** Following the active data acquisition strategy of [3], it actively selects object-specific

interactions from the available data pool by acquiring the most informative samples.

- **HACMAN [6]:** A model-free reinforcement learning method utilizing an actor-critic model, in which the critic learns to pick good contact points, and the actor learns the best action to take at each contact point. Unlike the other baselines, it directly trains a pushing policy instead of learning an explicit pushing model.
- **Planning Baselines**
 - **Learned Model + MPPI [22]:** Serves as a closed-loop controller, which samples candidate action sequences, propagates them through the current learned dynamics model, and evaluates them according to the defined manipulation objective.

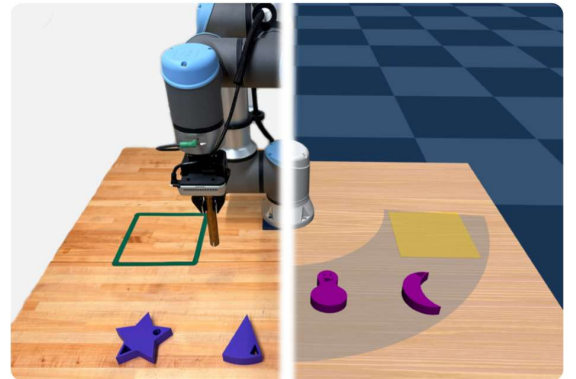


Fig. 5: **Physical and Simulation Setup for Evaluation.** The gray annular region shows $\mathcal{X}_{\text{free}}$ based on the robot’s safe workspace, the yellow region denotes the start region, and the goal region is on the opposite side in green. Moreover, the planner considers clearance as the safety margin, and MPPI penalizes trajectories that approach the boundaries.

A. Learning Performance

This section evaluates how rapidly each dynamics learning approach improves its predictions when interacting with a new object. In this experiment, we evaluate the dynamics model independently of the downstream manipulation task.

¹The discrete contact point representation in Sec. IV does not satisfy the assumptions required for asymptotic optimality of AORRT. Although the planner can generate a feasible long-horizon trajectory, in theory, the contact point can be chosen from a continuous boundary of the object.

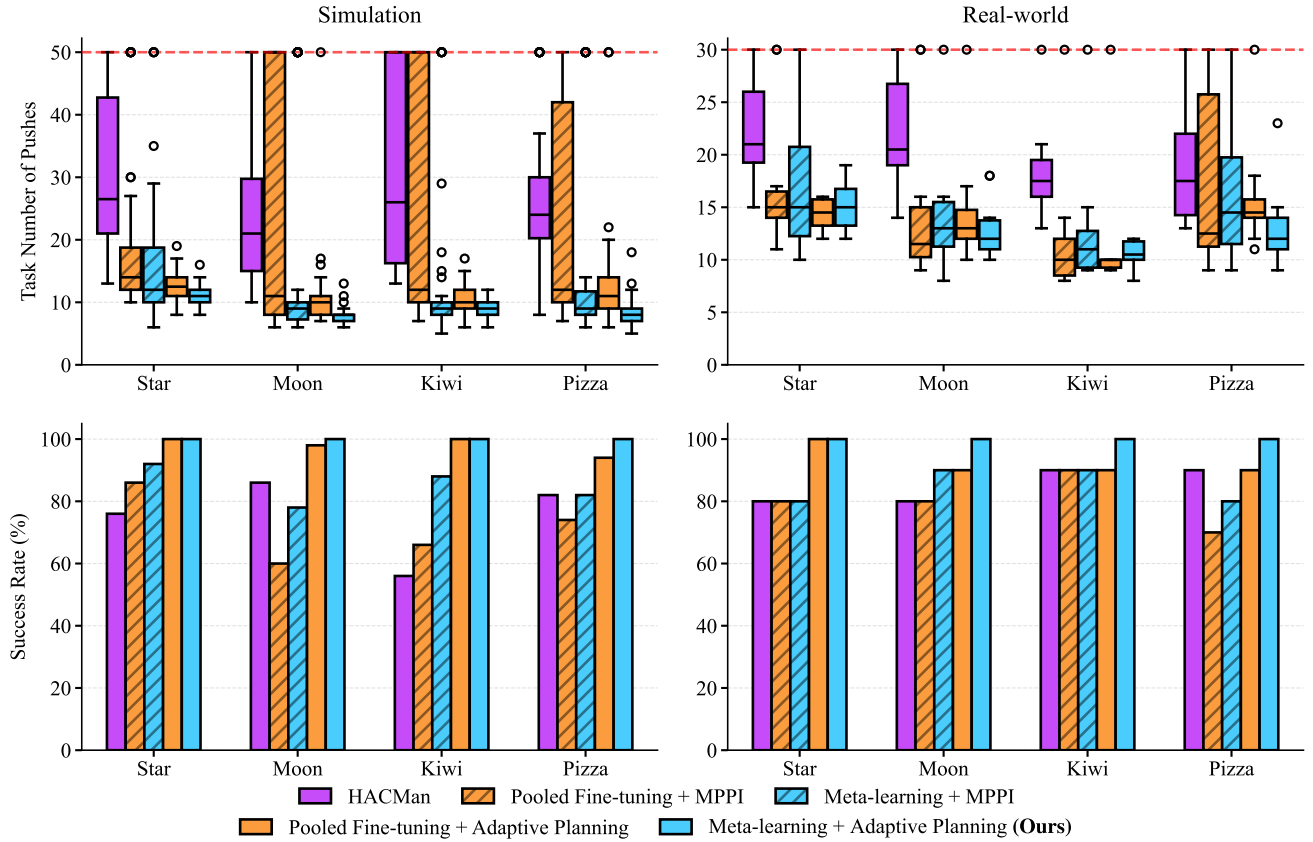


Fig. 6: **Push-to-Goal Manipulation Performance.** Number of pushes required to move the object to the goal across four unseen evaluation objects for 50 trials in simulation and 10 in the real world. Results compare POOLED FINE-TUNING and META-LEARNING with MPPI and ADAPTIVE PLANNING, together with HACMAN. A trial fails if the object does not reach the goal within 50 pushes in simulation or 30 pushes in the real-world.

We record 50 pushes for 10 different objects in simulation and on physical hardware. In simulation, we collect 2000 independent pushes, reserving 1000 as an evaluation set and using the remaining 1000 as the adaptation pool. For each physical object, the dataset includes 600 pushes, split into 200 for evaluation and 400 for adaptation.

Prediction accuracy is measured on a fixed evaluation set by comparing the predicted and ground-truth state using Eq. 7. META-LEARNING and POOLED FINE-TUNING randomly sample interactions from the adaptation pool set and update the model, whereas ACTIVE LEARNING starts by training on 10 data points and then selects informative interaction batches from the same pool.

As shown in Fig. 4, the proposed META-LEARNING approach achieves lower prediction error with substantially fewer interactions across both simulation and real-world settings. This method is explicitly trained to provide an initialization that adapts rapidly to unseen objects. In contrast, POOLED FINE-TUNING generally cannot provide reliable predictions during the first few interactions and typically requires more object-specific data to approach comparable prediction accuracy. For some objects, e.g., *Power Drill* and *Banana*, this difference remains visible throughout the evaluated interaction budget.

ACTIVE LEARNING behaves similarly to POOLED FINE-TUNING model. It has a larger prediction error initially, but

progressively improves with more interactions. Its accuracy eventually approaches the other two methods for several objects and achieves even lower error for *Letter T*. This demonstrates that ACTIVE LEARNING can produce accurate dynamics with sufficient single-object interactions, however, it trains a separate model from scratch for each new object and is not transferable to different geometries.

Importantly, although the meta-trained dynamics model is trained entirely from simulated push data, it provides an effective initialization for unseen object behavior in the real world. Despite differences in pushing dynamics from the simulated environment to the real world, our learned initialization adapts rapidly with only a limited number of physical interactions, demonstrating the method’s sim-to-real transfer capability.

B. Manipulation Performance

We next evaluate how online dynamics adaptation and our adaptive planning affect performance on a long-horizon manipulation task. Here, we consider four combinations of a learned model and a controller: POOLED FINE-TUNING with MPPI, META-LEARNING with MPPI, POOLED FINE-TUNING with ADAPTIVE PLANNING, and META-LEARNING with ADAPTIVE PLANNING. The final combination corresponds to the complete METAPUSHER framework.

Fig. 6 compares the number of pushes required to reach the goal and the corresponding task success rate of different

methods across the four evaluation objects in simulation and the real world. Overall, the results show that rapid dynamics model adaptation that improves manipulation performance, together with long-horizon planning that can exploit this improved model, yields a more efficient and robust trajectory toward the goal. The simulation results first demonstrate the effect of dynamics adaptation while keeping the controller fixed. Using MPPI, replacing the POOLED FINE-TUNING model with a META-LEARNING model increases the task success rate by **13.5%** (71.5% $\rightarrow$ 85.0%) and decreases the number of pushes required by **6.38** (22.52 $\rightarrow$ 16.14), as faster dynamics adaptation translates into more effective actions. A similar trend holds with ADAPTIVE PLANNING, where Meta-learning achieves a success rate of **100%** across all trials and reduces the number of pushes by **2.9** (11.99 $\rightarrow$ 9.09). These results are consistent with Sec. V-A, i.e., improving the dynamics model at early stages of interaction leads to more reliable predictions for manipulation.

Furthermore, Fig. 6 shows a substantial difference between MPPI and METAPUSHER’s ADAPTIVE PLANNING when the dynamics model is held fixed. With POOLED FINE-TUNING, ADAPTIVE PLANNING increases task success by **26.5%** (71.5% $\rightarrow$ 98%) compared to MPPI. Using META-LEARNING, the proposed planning strategy increases the success rates from **85%** for MPPI to **100%**. Unlike MPPI, which optimizes actions over a short receding horizon, the planner searches for a dynamically feasible trajectory extending to the goal. Moreover, as described in Fig. 3, METAPUSHER adapts the existing planning tree as the learned dynamics evolve, rather than replanning entirely from scratch.

The real-world experiment further evaluates the complete framework under perception noise, execution uncertainty, and sim-to-real dynamics differences. Despite the perception noise, execution uncertainties, and sim-to-real dynamics differences, METAPUSHER successfully completes the manipulation task in all evaluated trials across all four objects, achieving a **100%** success rate.

HACMAN, as a policy-based baseline, achieves lower success rates and generally requires more pushes than the model-based methods, due to the long-horizon nature of the task and the hard workspace boundaries. HACMAN’s struggles also stem from its inability to adapt to object-specific dynamics models during execution.

C. Training Analysis

We further evaluate how well METAPUSHER generalizes to unseen objects by comparing it with models that include evaluation objects during training. This comparison against the POOLED FINE-TUNING baseline here differs from the one in Sec. V-A and Sec. V-B. Here, this model is trained using $\mathcal{O}_{\text{train}} \cap \mathcal{O}_{\text{test}}$, while Object-specific training learns a separate model for each object using interaction data from only that object $o_i \in \mathcal{O}_{\text{test}}$. In contrast, METAPUSHER is meta-trained exclusively on the $\mathcal{O}_{\text{train}}$ set and has no interaction with $\mathcal{O}_{\text{test}}$ before the task. All methods use ADAPTIVE PLANNING, isolating the effect of prior experience on manipulation performance for the evaluation objects.

TABLE I: Task performance across four evaluation objects under different training-data regimes using ADAPTIVE PLANNING.

Object, i	POOLED FINE-TUNING	METAPUSHER	OBJECT-SPECIFIC MODEL
<i>Training data</i>	$\mathcal{O}_{\text{train}} \cap \mathcal{O}_{\text{test}}$	$\mathcal{O}_{\text{train}}$	o_i
<i>Cracker Box</i>			
Success Rate	100%	100%	100%
Number of Pushes	15.46	10.88	9.68
Tracking Error	0.0731	0.0324	0.0317
<i>Mustard Bottle</i>			
Success Rate	98%	100%	100%
Number of Pushes	14.10	9.52	10.36
Tracking Error	0.0934	0.0574	0.0950
<i>Letter T</i>			
Success Rate	100%	100%	100%
Number of Pushes	12.52	12.14	10.00
Tracking Error	0.0789	0.0545	0.0647
<i>Banana</i>			
Success Rate	98%	100%	100%
Number of Pushes	14.02	11.56	11.96
Tracking Error	0.101	0.068	0.074

As shown in Table I, METAPUSHER generalizes well despite not having access to interaction data of any evaluation object. Compared with POOLED FINE-TUNING, which has seen all four objects during training, our method requires fewer pushes per object and achieves consistently lower tracking error. Hence, METAPUSHER’s performance does not rely on including target objects in the training set.

Moreover, the proposed framework achieves performance comparable to Object-specific training, which is trained only on the same object it is evaluated on. Nevertheless, METAPUSHER achieves lower tracking error for *Mustard Bottle* (39.6% improvement), *Letter T* (15.8% improvement), and *Banana* (8.1% improvement), while remaining close on *Cracker Box*. Both the proposed framework and the OBJECT-SPECIFIC MODEL succeed on all trials across all four objects, with METAPUSHER taking fewer pushes for *Mustard Bottle* and *Banana*.

Overall, although Object-specific training has direct access to each evaluation object’s interaction data, our method can achieve comparable, and in several cases better, performance by adapting the dynamics model online during the task. These results show that the model’s meta-learned initialization is not specific to evaluation objects and is transferable to unseen objects.

VI. CONCLUSION

In this work, we present METAPUSHER, a framework for nonprehensile manipulation of previously unseen objects in simulation and the real world. METAPUSHER combines a geometry-based forward dynamics model, meta-training across diverse training objects, online dynamics adaptation from task-generated interactions, and adaptive kinodynamic planning. The meta-learned initialization enables the dynamics model to rapidly adapt to previously unseen objects from limited interaction data, while the planner retains and refines its existing search tree as the dynamics evolve.

METAPUSHER achieves lower prediction error than POOLED FINE-TUNING and ACTIVE LEARNING by combining META-LEARNING with ADAPTIVE PLANNING. For a long-horizon manipulation task, it improves task success rate and reduces the number of pushes compared with MPPI-based alternatives. Additionally, METAPUSHER demonstrates effective sim-to-real transfer by successfully completing all manipulation trials. Furthermore, results show that this method is not object-specific and can transfer to unseen objects while progressively incorporating their latent physical behavior during task execution.

VII. LIMITATIONS AND FUTURE WORK

The current framework has several limitations. The dynamics model relies on accurate contour point cloud observations and state estimation. Moreover, the current formulation considers planar manipulation in $SE(2)$ only by applying normal-direction push actions.

Future work will extend the framework to full $SE(3)$ manipulation and more complex contact interactions, and manipulation with multiple contact points. Incorporating force sensing at the end effector could also provide direct information about contact forces and friction.

REFERENCES

- [1] K. M. Lynch and M. T. Mason, "Stable pushing: Mechanics, controllability, and planning," *Intl. J. of Robotics Research*, vol. 15, no. 6, pp. 533–556, 1996. <https://journals.sagepub.com/doi/abs/10.1177/027836499601500602>
- [2] M. T. Mason, "Mechanics and planning of manipulator pushing operations," *Intl. J. of Robotics Research*, vol. 5, no. 3, pp. 53–71, 1986. <https://journals.sagepub.com/doi/abs/10.1177/027836498600500303>
- [3] Z. Zhong, S. Golestaneh, and C. Chamzas, "Activepusher: Active learning and planning with residual physics for nonprehensile manipulation," *IEEE Intl. Conf. Robot. Autom.*, 2025. <https://elplab.org/assets/pdf/zhong2026activepusheractivelearningplanning.pdf>
- [4] M. Bauza, F. R. Hogan, and A. Rodriguez, "A data-efficient approach to precise and controlled pushing," in *Conf. on Robot Learning*. PMLR, 2018, pp. 336–345. <https://proceedings.mlr.press/v87/bauza18a.html>
- [5] P. Agrawal, A. V. Nair, P. Abbeel, J. Malik, and S. Levine, "Learning to poke by poking: Experiential learning of intuitive physics," *Advances in Neural Information Processing Systems*, vol. 29, 2016. https://proceedings.neurips.cc/paper_files/paper/2016/file/c203d8a151612acf12457e4d67635a95-Paper.pdf
- [6] W. Zhou, B. Jiang, F. Yang, C. Paxton, and D. Held, "Hacman: Learning hybrid actor-critic maps for 6d non-prehensile manipulation," *Conf. on Robot Learning*, 2023. <https://hacman-2023.github.io/>
- [7] Y. Cho, J. Han, Y. Cho, and B. Kim, "Corn: Contact-based object representation for nonprehensile manipulation of general unseen objects," in *Intl. Conf. on Learning Representations*, 2024, pp. 46 130–46 146. https://proceedings.iclr.cc/paper_files/paper/2024/hash/c981fd12b1d5703f19bd8289da9fc996-Abstract-Conference.html
- [8] A. Nagabandi, I. Clavera, S. Liu, R. S. Fearing, P. Abbeel, S. Levine, and C. Finn, "Learning to adapt in dynamic, real-world environments through meta-reinforcement learning," *Intl. Conf. on Learning Representations*, 2019. <https://openreview.net/forum?id=HyztsoC5Y7>
- [9] Z. Xian, S. Lal, H.-Y. Tung, E. A. Platanios, and K. Fragkiadaki, "Hyperdynamics: Meta-learning object and agent dynamics with hypernetworks," *Intl. Conf. on Learning Representations*, 2021. <https://openreview.net/forum?id=pHXfe1cOmA>
- [10] J. K. Li, W. S. Lee, and D. Hsu, "Push-net: Deep planar pushing for objects with unknown physical properties," in *Robotics: Science and Syst.*, vol. 14, 2018, pp. 1–9. <https://www.roboticsproceedings.org/rss14/p24.pdf>
- [11] J. Lyu, Z. Li, X. Shi, C. Xu, Y. Wang, and H. Wang, "Dywa: Dynamics-adaptive world action model for generalizable non-prehensile manipulation," in *IEEE/CVF Intl. Conf. on Computer Vision*, 2025, pp. 11 058–11 068. <https://ieeexplore.ieee.org/abstract/document/11445531>
- [12] A. Orthey, C. Chamzas, and L. E. Kavraki, "Sampling-based motion planning: A comparative review," *Annual Review of Control, Robot., and Autom. Syst.*, vol. 7, 2024. <https://www.annualreviews.org/content/journals/10.1146/annurev-control-061623-094742>
- [13] A. Sivaramakrishnan, S. Tangirala, D. M. Ramesh, E. Granados, and K. E. Bekris, "Kraft: Sampling-based kinodynamic replanning and feedback control over approximate, identified models of vehicular systems," *arXiv preprint arXiv:2409.11522*, 2024. <http://arxiv.org/abs/2409.11522>
- [14] S. Golestaneh, Z. Zhong, D. Lee, and C. Chamzas, "Aura: Asymptotically optimal uncertainty-robust replanning algorithm for kinodynamic systems," *arXiv preprint arXiv:2605.27699*, 2026. <https://arxiv.org/abs/2605.27699>
- [15] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *Intl. Conf. on Machine Learning*. PMLR, 2017, pp. 1126–1135. <https://proceedings.mlr.press/v70/finn17a.html>
- [16] K.-T. Yu, M. Bauza, N. Fazeli, and A. Rodriguez, "More than a million ways to be pushed: a high-fidelity experimental dataset of planar pushing," in *IEEE/RSJ Intl. Conf. on Intell. Robots and Syst.* IEEE, 2016, pp. 30–37. <https://ieeexplore.ieee.org/abstract/document/7758091>
- [17] B. Jiang, Y. Wu, W. Zhou, C. Paxton, and D. Held, "Hacman++: Spatially-grounded motion primitives for manipulation," *Robotics: Science and Syst.*, 2024. <https://sgmp-rss2024.github.io/>
- [18] H. Bui, Y. Gao, H. Yang, E. Cui, S. Mody, B. Acosta, T. S. Felix, B. Bianchini, and M. Posa, "Push anything: Single- and multi-object pushing from first sight with contact-implicit mpc," *IEEE Intl. Conf. Robot. Autom.*, 2026. <https://dairlab.github.io/push-anything/>
- [19] J. Levy, J. Gibson, B. Vlahov, E. Tever, E. Theodorou, D. Fridovich-Keil, and P. Spieler, "Meta-learning online dynamics model adaptation in off-road autonomous driving," *Robotics: Science and Syst.*, 2025. <https://openreview.net/forum?id=jnKYdHfFWA>
- [20] L. Cong, M. Grner, P. Ruppel, H. Liang, N. Hendrich, and J. Zhang, "Self-adapting recurrent models for object pushing from learning in simulation," in *IEEE/RSJ Intl. Conf. on Intell. Robots and Syst.*, 2020, pp. 5304–5310. <https://ieeexplore.ieee.org/abstract/document/9341076>
- [21] J. Richalet, A. Rault, J.-L. Testud, and J. Papon, "Model predictive heuristic control," *Automatica*, vol. 14, no. 5, pp. 413–428, 1978.
- [22] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, "Aggressive driving with model predictive path integral control," in *IEEE Intl. Conf. Robot. Autom.*, 2016, pp. 1433–1440. <https://ieeexplore.ieee.org/abstract/document/7487277>
- [23] N. Perrault, Q. H. Ho, and M. Lahijanian, "Kino-pax: Highly parallel kinodynamic sampling-based planner," *IEEE Robot. Autom. Letters*, vol. 10, no. 3, pp. 2430–2437, 2025. <https://ieeexplore.ieee.org/abstract/document/10844318>
- [24] Y. Gao, A. Lu, and Z. Kingston, "Fast asymptotically optimal kinodynamic planning via vectorization," *IEEE/RSJ Intl. Conf. on Intell. Robots and Syst.*, 2026. <https://arxiv.org/abs/2607.03987>
- [25] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "Pointnet: Deep learning on point sets for 3d classification and segmentation," in *IEEE Conf. on Computer Vision and Pattern Recognition*, 2017, pp. 652–660. https://openaccess.thecvf.com/content_cvpr_2017/html/Qi_PointNet_Deep_Learning_CVPR_2017_paper.html
- [26] B. Calli, A. Singh, A. Walsman, S. Srinivasa, P. Abbeel, and A. M. Dollar, "The ycb object and model set: Towards common benchmarks for manipulation research," in *Intl. Conf. on Advanced Robotics (ICAR)*, 2015, pp. 510–517. <https://ieeexplore.ieee.org/document/7254318>
- [27] L. Downs, A. Francis, N. Koenig, B. Kinman, R. Hickman, K. Reymann, T. B. McHugh, and V. Vanhoucke, "Google scanned objects: A high-quality dataset of 3d scanned household items," in *IEEE Intl. Conf. Robot. Autom.*, 2022, pp. 2553–2560. <https://ieeexplore.ieee.org/abstract/document/9811809>
- [28] K. Hauser and Y. Zhou, "Asymptotically optimal planning by feasible kinodynamic planning in a state-cost space," *IEEE Trans. Robot.*, vol. 32, no. 6, pp. 1431–1443, 2016. <https://ieeexplore.ieee.org/abstract/document/7588078>
- [29] I. A. Sucan, M. Moll, and L. E. Kavraki, "The open motion planning library," *IEEE Robot. Autom. Magazine*, vol. 19, no. 4, pp. 72–82, 2012. <https://ompl.kavrakilab.org>
- [30] B. Wen, W. Yang, J. Kautz, and S. Birchfield, "Foundationpose: Unified 6d pose estimation and tracking of novel objects," in *IEEE Conf. on Computer Vision and Pattern Recognition*, 2024, pp. 17 868–17 879. <https://ieeexplore.ieee.org/abstract/document/10655554>